LMS Side Project – Short Overview

What this is - “Moodle für Arme”

We are considering building a small Learning Management System (LMS) as a side project. Not to outdo existing systems like the one we use as RWTH students (Moodle), but simply as a learning project to deliberately practice full-stack web development by recreating the core ideas of such a system from scratch.

An LMS, in simple terms, is a web application that allows:

- User accounts with different roles (e.g. students, instructors, admins)
- Creation and organization of courses
- Uploading and accessing course materials
- Assignments and submissions
- Basic grading or feedback
- Possibly discussions or messaging

We would start with a very minimal version and gradually expand it.

Why we want to do it

The main motivation is structured practice.

Instead of isolated exercises or tutorials, this would be a shared project where we:

- Design and evolve an application architecture
- Implement frontend and backend components
- Build and consume APIs
- Implement authentication and authorization
- Integrate some form of persistent data storage
- Practice collaboration via Git workflows
- Work in small, defined iterations

The goal is not necessarily speed, but depth and consistency.

How we imagine working

The project is open-ended, but structured.
We would work in short sprints (depending on the sprint goal, say, 1–2 weeks).

For each sprint:

- Agree on a concrete goal (a working increment)
- Split responsibilities

- Implement and review
- Merge and stabilize

The idea is that this is a good way of getting things done and also practice for the real world. Depending on the situation of the team members and their other responsibilities like studies, work etc., there may be pauses between sprints, or perhaps some sprints with less content, or perhaps a sprint where one team member does not partake at all. The idea is that we will plan the sprints together in a meeting and sort these things out. Also, we might define core and optional tasks for each team member for each sprint, and record these on a lightweight sprint document or something of that nature.

The overall work is obviously voluntary (none of us is getting paid to do this), but during an agreed sprint we should treat responsibilities seriously. Without defined goals and timelines, the project will likely stall — so we want at least lightweight structure.

Possible Areas of Responsibility

This is not fixed, but roughly:

Backend

- Application logic
- API design
- Authentication and role handling
- Data storage integration (technology to be decided)
- Validation and testing

Frontend

- User interface and routing
- Forms (login, course creation, submissions, etc.)
- API integration
- State management
- Basic UX consistency

General / Shared

- Architecture decisions
- Code review
- Documentation
- Sprint planning

No specific tech stack is fixed yet. Part of the learning process is deciding this together. Obviously it would make sense to pick those that are most widely used as of today.

Tools

We would mainly use:

- **GitHub** for version control, issues, and reviews
- **Discord** for communication and coordination

Project Language

We are all German-speaking Computer Science students at RWTH Aachen University who are also fluent in English. Since English is kind of the universal language in the technology world it probably makes sense to use it for documentation like this and maybe throw in bits of German here and there. Obviously also in the meetings we might speak German.

Current Team Members

- **Maxim Biliiciuc**
- **Johannes Lürgen**

Notes

- Note by team member Johannes Lürgen:
  Ich denke ich würde tendenziell gerne frontend responsibilites übernehmen da ich zuletzt innerhalb des RWTH Software Projektpraktikums Backend gemacht habe und ich somit meine web development erfahrungen komplementieren könnte

(This document has been AI-prompted and then extensively edited by Johannes Lürgen)